

Deep Learning

Exercise 1: Python

Room: **BIN-1-B.01**

Instructor: Manuel Günther

Email: guenther@ifi.uzh.ch

Office: AND 2.54

Friday, February 20, 2026, 10:15 – 12:00

Outline

- 1 Details of the Exercise
- 2 Python

Structure of Assignments

General Structure

- Each assignment is provided as Jupyter notebook
- They include three parts each

Part 1: Task Description

- Description of the problem to be solved
- Details of the tasks and possible solutions
- Possible outcomes of plotting functionality
 - Provides hints how solutions could look like

Details of the Exercise

Part 2: Theory

- Theoretical problem analysis
- Design of a solution and selection of parameters
- Mathematical derivations
- **prepares for theoretical part of the exam**
- The theoretical part is **not graded**

Details of the Exercise

Part 3: Coding

- Code blocks with template code and "..."
- Typically, functions need to be implemented
- Test cases assure that the function returns are correct
- Plotting functions are mostly provided
- Run blocks sequentially to obtain output
- Code is **graded with pass/fail** grade
 - We are generally generous
- Partial solutions are OK
 - at least half of coding tasks need to be addressed

Details of the Exercise

Structure and Requirements of Assignments

- Working in teams (1-3 students)
 - List team members in Jupyter notebook
- Submit code solution on OLAT
 - Deadline: Wednesday, 23:59
 - One submission per team is sufficient
- We discuss your code the following Friday
- Feedback follows the next week(s)
 - Share feedback with the team members
- 3 successful submissions (A1-A5) required for exam
- 5 bonus points for each additional assignment

Details of the Exercise

Schedule of Assignments and Mock Exam

#	handout	theory	submission	discussion
A1	Feb 27	Mar 6	Mar 11	Mar 13
A2	Mar 13	Mar 20	Mar 25	Mar 27
A3	Mar 27	Apr 17	Apr 22	Apr 24
A4	Apr 24	May 8	May 13	May 15
A5	May 15	May 22	May 27	May 29
Mock	May 22	—	—	May 29

Jupyter Notebooks

Jupyter Notebooks

- Combination of text (explanation) and code
 - Markdown markup language including headers, math, hyperlinks, ...
- Allows easy interaction and debugging by running separate cells
- Integrated in many editors (e.g. Visual Studio Code)
- Online documentation available <http://docs.jupyter.org>
- Supported by Google Colab <http://colab.research.google.com>
- Drawback: not useful for larger projects

Assignment Support

All assignments are accompanied by a Jupyter notebook template

Optional Local Installation

Conda Installation

- Install Conda for your operating system:
<http://docs.conda.io/en/latest/miniconda.html>
- Optional: create and activate local environment
 - Create environment: `conda create -name DL python=3.10`
 - Activate environment: `conda activate DL`
- Install packages
 - `conda install numpy scipy scikit-learn matplotlib jupyter`
- Run from console (opens in web browser): `jupyter notebook`
<https://realpython.com/jupyter-notebook-introduction>
- Visual Studio Code setup:
<https://code.visualstudio.com/docs/datascience/jupyter-notebooks>

Data Types and Operations

Dynamically Typed Numbers

- Boolean:
 - `k = True`
- (Signed) integer:
 - Simple: `a = -42`
 - Long: `b = 12345678901234567890`
 - Hex: `c = 0x6AfEb5`
- Float (IEEE 754 double):
 - Simple: `pi = 3.14159265`
 - Scientific: `sci = 1.4142e-5`
- Complex numbers (128 bit):
 - Simple: `cplx = -1.5 + 3.2j`

Operations

- Boolean: `not`, `and`, `or`
- Numerical: `+`, `-`, `*`, `/`, `//`, `%`, `**`
 - `5/2 = 2.5`, but `5//2 = 2`
 - Modulo: `24 % 10 = 4`
 - Power: `5 ** 2 = 25` `not` `^`
- Assignment: `=`, `+=`, `-=`, `/=`, `//=`, `%=`, `**=`
- Comparison: `<`, `<=`, `>`, `>=`, `==`, `is`, `in`
 - Returns Boolean
 - Operator Chaining: `a < b < c`
- Conversion:
 - `float(b)`, `int(pi)`

Datatypes in Python

Complex Datatypes

- String (immutable):
 - `s1 = "Hello"`
 - `s2 = 'world'`
 - `s3 = """Multiline
string with
exact interpretation"""`
- List, Tuple (immutable)
 - `x = [-1e-5, 'text', s1]`
 - `y = (s2,)`
- Dictionary
 - `d = {"a" : 0.4, 17 : c}`
- Nonetype `None`

Operations

- Concatenation:
 - String: `"Hello" + ' world!'`
 - List: `x + [y]` and `x += [y]`
 - Tuples and lists: ~~`[1] + (2,)`~~
- Multiplication:
 - String: `"ab"*4`
 - List/Tuple: `[2]*10, ('o',)*10`
- Indexing:
 - Dictionary: `d['a'], d[1.5]='x'`
 - List/Tuple: `y[0], x[2]=None`
 - Range: `x[1:3], x[:2], x[2:]`
 - Negative: `x[-1], x[-2:]`

List Manipulation

- ## Example

```
# delete values
data.remove("three")
data.pop(1)
data.pop(-2)
del data[3]
```

Datatypes in Python

Type checking via `type`

- `type(1) → <class 'int'>`
- `type(1.) → <class 'float'>`
- `type(1j) → <class 'complex'>`
- `type({}) → <class 'dict'>`
- `type(()) → <class 'tuple'>`
- `type((1)) → <class 'int'>`
- `type((1,)) → <class 'tuple'>`
- `type([]) → <class 'list'>`
- `type("") → <class 'str'>`

Type testing via `isinstance`

- `isinstance(1, int) → True`
- `isinstance(1, float) → False`
- `isinstance(1., float) → True`
- `isinstance(1., (int, float)) → True`
- `isinstance('', str) → True`
- `isinstance([], str) → False`
- `isinstance((1), tuple) → False`
- `isinstance((1,), tuple) → True`

Control Flow

Indentation

- Blocks have same indentation
- Default: 4 spaces
- Mixing tabs and spaces
 - Strongly discouraged
 - Setup editor to insert spaces

if Statements

```
if Boolean Condition:  
    Code for True Condition  
else:  
    Optional Code for False Cd.
```

Control Flow

while Loops

```
while Boolean Condition:
    if Skip Condition:
        continue
    Repeated Code
    if Additional Condition:
        break
else:
    Code when exited normally
```

while Loop Example

```
data = [0, [1], "two", 3., '4', -5]
index = 0
while index < len(data):
    if isinstance(data[index], list):
        continue

    number = float(data[index])
    print (number*2)
    index += 1

    if data[index] < 0:
        break

else:
    print ("Success!")
```

Control Flow

for Loops

```
for item in iterable:
    if Skip Condition:
        continue
    Code using item
    if Additional Condition:
        break
else:
    Code when exited normally
```

Iterables and Examples

- Lists/Tuples
 - Elements: `for e in [1,4,7,10]`
- Ranges:
 - `for i in range(0,20,2)`
 - ⇒ start, end (exclusive), stepwidth
- Enumerations:
 - `for i, e in enumerate([1,4,7,10])`
 - ⇒ Tuple Unboxing
- Dictionaries
 - Keys: `for k in {1:2, 3:4}`
 - Both: `for k, v in {1:2, 3:4}.items()`

Control Flow

List Comprehensions

- Turn one list into another
- Filter elements (optional)
- Nested List Comprehensions

Definition

```
[f(x) for x in src if cond(x)]
```

Examples

```
squares = [x**2 for x in range(10)]  
odds = [x for x in range(10) if x % 2]  
  
triang = [[x for x in range(10) if x >= y]  
          for y in range(10)]
```

Dictionary Comprehensions

- Create dictionaries
- Filter elements (optional)
- Nested comprehensions

Definition

```
{key : val for ... if ...}
```

Examples

```
squares = {x : x**2 for x in range(10)}  
halves = {x : x//2 for x in range(10)  
          if x % 2}  
  
inv = {v : k for k, v in {...}.items()}
```

Functions

Function Definition

```
def func_name(arguments, keywords):  
    Code  
    return ...
```

Arguments

- Required to be specified
- No datatype is required

Keywords

- Arguments with default values
→ Syntax: `name=value`
- **Warning: default values `[]` or `{}`**

Function Call Examples

```
def func1(data, opt=None):  
    if opt is None:  
        print("data is", data)  
    else:  
        return data
```

```
func1(27)  
→ prints "data is 27"  
→ returns None
```

```
func1(27, True)  
→ returns 27
```

```
func1(opt = False, data = 27)  
→ returns 27
```

Functions

Lambda Functions

- Inline function w/o name
- Single-line function only
- List of arguments
- No keyword arguments
- Directly returns result
- Often used in `apply` or `map`

Function Definition

```
func = lambda arguments: ...  
res = func(...)
```

Lambda Example

```
add_five = lambda x: x+5  
print(add_five(3))
```

```
hypotenuse = lambda a,b: \  
    math.sqrt(a**2 + b**2)  
print(hypotenuse(3,4))
```

```
data = [1,2,3]  
negs = map(lambda x: -x, data)  
print(list(negs))
```

Functions

Generator Function

- To be used, e.g., with `for` loop
- Returns samples one by one
→ Useful for large collections

Function Definition

```
def func_name(arguments, keywords):  
    while ....  
        yield ...
```

Predefined Generators

- `range`, `enumerate`, `map`
- `dict.keys()`, `dict.items()`

Generator Example

```
def sup(it1, it2):  
    for v1 in it1:  
        for v2 in it2:  
            yield v1, v2  
  
for t in sup("ab", range(2)):  
    print(t)  
  
( 'a', 0)  
( 'a', 1)  
( 'b', 0)  
( 'b', 1)
```

Classes and Objects

Function Definition

```
class ClassName(parents):  
    member = None  
  
    def __init__(self, params):  
        super(ClassName, self).__init__()  
        Initialization of self  
  
    def method(self, params):  
        Can use self and params  
        return ...  
  
    @staticmethod  
    def static(params):  
        Cannot use self  
        return ...
```

Members

- Prior definition optional
- Access through `self.member`
- Publicly accessible

Constructor `__init__`

- Base class constructor
- Initializes object (`self`)

Functions

- First parameter: `self`
- Special function `__call__`

Classes and Objects

Example

```
class AddN(object):
    def __init__(self, n=1):
        super(AddN, self).__init__()
        self.n = n

    def __call__(self, x):
        return x + self.n

    def squared(self, x):
        return x**2 + self.n

    def sequence(self, x):
        for n in range(self.n):
            yield x + n
```

Example (Contd.)

```
add1 = AddN(n=1)
print(add1.squared(5))
print(add1(5))

add1.n = 4
add1.unused = None
print(add1.squared(5))

for n in add1.sequence(10):
    print(n)
```

Classes and Objects

Objects in Python

In Python (almost) everything is an object

Immutable Objects

```
def modify(x):  
    x += 1
```

```
y = 10  
modify(y)  
print(y) -> 10
```

Mutable Objects

```
def modify(x):  
    x += [1]  
  
y = [10]  
append(y)  
print(y) -> [10,1]
```

Class Objects

```
my_int = int  
my_int("42") -> 42
```

Non-Objects

Keywords like
for, in, def, class

Functions are Objects

```
def sqr(x):  
    return x**2  
  
def apply(f, data):  
    return f(data)  
  
print(apply(sqr, 5))  
print(apply(add1, 5))
```

Exceptions

Exception Handling

```
try:  
    Code  
  
except Exception1 as ex:  
    print(ex)  
    raise Exception() from ex  
  
except (Exception2, Exception3):  
    Handle exception  
    raise  
  
finally:  
    Always executed
```

Types of Exceptions

- Base class: `Exception`
- `SyntaxError`
- `ArithmeticError`
- `ValueError`
- `IndexError`, `KeyError`
- `MemoryError`
- `StopIteration`, `SystemExit`
- Own Exceptions
→ derive from `Exception`

String Formatting

Old Types of String Formatting

- The `%` syntax:
 - `"name: %s; age: %d" % ("John", 42)`
- The `format` syntax:
 - `"name: {}; age: {}".format("John", 42)`
 - `"name: {name}; age: {age}".format(age=42, name="John")`

Python `f`-Strings

- Special syntax `f"""` or `F"""`
- Evaluates expressions in `{}`
- Number formatting `f"{n:w.p}"`
 - `number`, `width`, `precision`

Example

```
data = {"name" : "John", "age" : 42}
print(f"name: {data['name']}; age: {data['age']}")

x = 3.14159265
print(f"pi = {x: 3.4}")
```

File I/O

The `open` function

- Signature: `open(name, mode, ...)`
- Filenames `"C:/path/to/file.txt"`
- Opening modes:
 - `'r'`ead, `'w'`rite, `'a'`ppend
 - `'t'`ext: `str`, `'b'`inary: `bytes`

The `with` Statement

- Ensures closure of object
- Syntax for files:
`with open(...) as file:`

Example

```
# write file
with open("test.txt", 'wt') as file:
    for x in range(1,11):
        file.write(f"{x},{x**2}\n")

# read file
with open("test.txt", 'rt') as file:
    for line in file:
        x, y = line.rstrip().split(',')
        print(x, y)
```